# STUDENT ATTENDANCE MANAGEMENT SYSTEM

*A Web-Based Attendance Management System Without Face Recognition*

**PROJECT REPORT**

Submitted in partial fulfilment of the requirements for the degree of

**BACHELOR OF COMPUTER APPLICATIONS (BCA)**

Submitted By

Name: _____

Roll No.: _____

Semester: _____

Under the Guidance of

_____

Department of Computer Applications

College: _____

Academic Session: _____

## DECLARATION

I hereby declare that the project report entitled “Student Attendance Management System” is an original academic project report prepared by me as part of the requirements for the Bachelor of Computer Applications (BCA) programme. The work presented in this report is based on the project scope, design, modules and technologies described herein. Any external material used for reference has been acknowledged in the References section.

Student Name: _____
Roll Number: _____
Class / Semester: _____
College: _____
Session: _____
Signature: _____

## CERTIFICATE

This is to certify that the project report entitled “Student Attendance Management System” has been prepared by _____, Roll No. _____, student of BCA, under the guidance of _____. The project report is submitted in partial fulfilment of the academic requirements of the Bachelor of Computer Applications programme.

Project Guide: _____
Head of Department: _____
Date: _____
College Seal: _____

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members and the Department of Computer Applications for their guidance and support during the planning and preparation of this project. I am also thankful to my college for providing the academic environment and resources required to understand software development concepts.

I would also like to thank my classmates, friends and family members for their encouragement. Finally, I acknowledge the documentation and learning resources of Python, Streamlit and Firebase, which helped me understand the technologies proposed for this system.

## ABSTRACT

The Student Attendance Management System is a simple web-based application designed to reduce the effort involved in maintaining attendance registers manually. Students can register and log in to mark their attendance for a selected subject. A Class Representative (CR) reviews pending attendance entries and verifies them after checking the class. Teachers can view attendance summaries, subject-wise records, student-wise percentages and reports through a dashboard.

The system is designed without face recognition or biometric hardware. It uses role-based access, a class-time window, duplicate-entry prevention and CR verification to discourage proxy attendance. Firebase Authentication and Cloud Firestore can be used for authentication and data storage, while Streamlit provides the user interface. The system can also generate defaulter lists and PDF/Excel reports. Optional AI-based insights may identify attendance trends from stored records.

The proposed solution is suitable as a small academic project because it uses free or free-tier tools and can be developed incrementally. Its safeguards reduce common misuse but cannot guarantee a student's physical presence; institutional supervision and CR verification remain important.

# TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Limitations
5. Existing and Proposed System
6. Feasibility Study
7. Technology Stack and Tools
8. System Requirements
9. System Architecture
10. Database Design
11. Module Description
12. System Workflow and Algorithms
13. Security and Anti-Proxy Measures
14. User Interface Design
15. Testing and Test Cases
16. Reports and AI Insights
17. Deployment Plan
18. Expected Results
19. Future Enhancements
20. Conclusion
21. References
22. Appendix A – Sample Firestore Structure
23. Appendix B – Sample Python Logic
24. Appendix C – Viva Questions and Answers

# 1. INTRODUCTION

Attendance is an important part of academic administration. In many institutions, attendance is recorded manually in registers or spreadsheets. This process takes time, makes report preparation difficult and can lead to calculation errors, missing entries or duplicate records. Teachers may also need to calculate attendance percentages separately for every student and subject.

The Student Attendance Management System aims to digitise this process through a web application. Students use their own login to submit attendance, the CR verifies the pending entries, and teachers use a dashboard to review attendance. The system stores the date, subject, student roll number, attendance status, marking time and verification details.

The project deliberately excludes face recognition and biometric devices. Instead, it uses practical controls such as a limited attendance window, one entry per student per subject per day, authenticated accounts and manual CR verification.

# 2. PROBLEM STATEMENT

Manual attendance systems can be slow and difficult to maintain. Registers may be misplaced or damaged, totals must be calculated manually, and preparing defaulter lists can consume considerable time. In a self-marking digital system, there is also a risk that one student may attempt to mark attendance for another student.

The problem addressed by this project is to provide a low-cost and easy-to-use attendance platform that simplifies record keeping while introducing basic checks against duplicate and proxy entries. The system should provide transparent verification and useful reports for teachers.

## 3. OBJECTIVES

• Provide student registration and secure login.
• Allow an authenticated student to select a subject and mark Present or Absent.
• Restrict attendance marking to the permitted class-time window.
• Prevent duplicate attendance entries for the same student, subject and date.
• Provide a CR panel to verify attendance after checking the class.
• Allow the CR to record an absent student when required.
• Provide teachers with date, subject and semester filters.
• Calculate total, present, absent and attendance percentage.
• Highlight students below the configurable attendance threshold.
• Generate PDF and Excel reports and optionally email a defaulter list.
• Explore AI-generated summaries of attendance patterns.

## 4. SCOPE AND LIMITATIONS

Scope: The application is intended for a college class or department. It supports student, CR and teacher roles; subject-based daily attendance; verification; dashboards; reports; and optional AI summaries. The initial project can be tested with a small group of students and a limited number of subjects.

Limitations: The application does not use face recognition, fingerprint scanning or dedicated biometric devices. A login or browser session alone cannot prove physical presence. GPS/browser location is optional and may be inaccurate or unavailable; it should not be treated as definitive proof. Email OTP depends on a correctly configured email service. Internet access is required for a cloud-hosted version. AI insights may be incomplete and should be reviewed by a teacher.

## 5. EXISTING AND PROPOSED SYSTEM

Existing System: Attendance is commonly recorded on paper or in separate spreadsheets. Teachers manually total attendance, calculate percentages and prepare defaulter lists. Searching historical records and correcting mistakes can be time-consuming.

Proposed System: A centralised web application stores attendance records in a cloud database. Students submit entries, CRs verify them, and teachers view consolidated dashboards. Automatic calculations and report generation reduce repetitive work and improve access to records.

Benefits include faster reporting, fewer calculation errors, centralised records, a clear verification status and easier identification of low-attendance students.

## 6. FEASIBILITY STUDY

Technical Feasibility: Python and Streamlit can be used to create the interface. Firebase Authentication and Cloud Firestore provide managed authentication and database services. ReportLab, pandas and Plotly support PDF reports, Excel export and charts.

Economic Feasibility: The prototype can be developed using free/open-source software and free-tier cloud services, subject to provider quotas and current pricing. A paid plan may be needed for larger usage or production deployment.

Operational Feasibility: The interface uses familiar forms, dropdowns and tables. Students need basic login skills; CRs and teachers need short training on verification and report filters.

Schedule Feasibility: A basic prototype can be built in stages: database and authentication, student flow, CR verification, teacher dashboard, reports, testing and deployment. A 15-hour estimate may be possible for a very small demonstration, but a reliable tested submission generally requires additional time.

## 7. TECHNOLOGY STACK AND TOOLS

Python: Main programming language for application logic.
Streamlit: Framework for building the web interface and dashboards.
Firebase Authentication: User registration and login.
Cloud Firestore: Cloud NoSQL database for student, subject and attendance data.
Pandas: Data processing and Excel export.
Plotly: Daily trends and subject-wise charts.
ReportLab: PDF report generation.
Gemini API (optional): Natural-language summaries of attendance data.
Gmail API / SMTP (optional): Emailing reports or OTP messages.
GitHub: Source-code version control.
Streamlit Community Cloud: Hosting option for the demonstration application.

All external services must be configured using secrets or environment variables. Firebase service-account keys, API keys and email credentials must never be committed to a public GitHub repository.

## 8. SYSTEM REQUIREMENTS

Hardware: A laptop/desktop for development; internet connection; and a modern smartphone or computer browser for users. A server or cloud account is required for hosted use.

Software: Windows, Linux or macOS; Python 3.x; Streamlit; Firebase project; a modern browser; Git; and the Python packages listed in requirements.txt.

User Roles: Student – registers, logs in and submits attendance. CR – views pending records, checks the class and verifies entries or records absences. Teacher – views dashboards, filters records and generates reports. Administrator – configures users, subjects, roles and access permissions (recommended for a complete deployment).

## 9. SYSTEM ARCHITECTURE

The system follows a simple client–application–database architecture. Users access the Streamlit interface through a browser. The application checks authentication and role permissions, applies attendance rules, and reads/writes records in Cloud Firestore. Reporting libraries prepare charts, PDF files and Excel files. Optional AI and email services are called only when configured.

Data flow: Student / CR / Teacher → Streamlit user interface → Authentication and role checks → Attendance business logic → Firestore → Dashboard, reports and optional insights.

The server must validate important rules. Hiding a button in the interface is not sufficient security; duplicate checks, time-window checks and role permissions must be enforced in trusted application logic and, where applicable, Firestore Security Rules.

## 10. DATABASE DESIGN

Collection: students
Fields: roll (unique identifier), name, course, semester, email, authUid, role, createdAt. Passwords should not be stored in this collection; Firebase Authentication manages credentials.

Collection: subjects
Fields: subjectId, subjectName, teacherName, semester, active.

Collection: attendance
Fields: attendanceId, date (YYYY-MM-DD), subjectId, roll, status (Present/Absent), markedTime, verified (true/false),

verifiedBy, verifiedAt, createdBy, source.

Recommended uniqueness rule: use a deterministic document ID such as date_subjectId_roll so a second record cannot be created for the same student, subject and date. The write should be transaction-safe to avoid simultaneous duplicate submissions.

Optional collection: users or profiles may store role and profile information linked to Firebase Authentication UID. Apply least-privilege database rules so students can only submit their own records and CR/teacher permissions are restricted to authorised classes.

## 11. MODULE DESCRIPTION

11.1 Registration Module: A Streamlit form collects roll number, name, course, semester, email and password. The application validates required fields, checks that the roll number is not already registered, creates the Firebase Authentication account and saves the profile. Passwords are never stored as plain text.

11.2 Login Module: The user signs in using the configured authentication method. The application loads the profile and directs the user to the correct role-based dashboard. Invalid credentials produce a safe error message.

11.3 Student Attendance Module: The dashboard displays the current date and available subjects. The student selects a subject and status, then submits attendance. The system validates the time window and checks for an existing record before saving a pending entry.

11.4 CR Verification Module: The CR signs in with an authorised CR account. A pending-verification table displays date, subject, roll number, student name, status and marking time. After physically checking the class, the CR verifies the entry. The CR may record an absence manually with an audit trail.

11.5 Teacher Dashboard: Teachers filter by date range, subject and semester. Summary cards show total students, present, absent and percentage. Tables show subject-wise and student-wise attendance. Colour bands can be used as visual indicators: red below 75%, yellow from 75% to 80%, and green above 80%, with threshold settings configurable by the institution.

11.6 Reports Module: The application creates a defaulter list when overall attendance or subject attendance is below the configured threshold. PDF reports can include roll number, student name, subject, present count, total classes, percentage and status. Excel export supports further analysis.

11.7 AI Insights Module (Optional): An AI service may summarise anonymised or minimum-necessary attendance aggregates, such as students below threshold, subjects with lower attendance and weekday patterns. The output is advisory, must be checked by staff and should not make disciplinary decisions automatically.

## 12. SYSTEM WORKFLOW AND ALGORITHMS

Student workflow: (1) Open application. (2) Register or log in. (3) Open student dashboard. (4) Select subject. (5) Select Present or Absent. (6) Submit within the allowed window. (7) Application checks identity, role, time and duplicate record. (8) Record is saved as pending verification. (9) CR reviews and verifies it.

CR workflow: (1) Log in with CR role. (2) Open pending-verification list. (3) Check the physical class register / students present. (4) Verify valid entries or mark absent where appropriate. (5) Save verifier identity and timestamp.

Teacher workflow: (1) Log in with teacher role. (2) Select date range, subject and semester. (3) View summary cards, tables and charts. (4) Generate PDF or Excel report. (5) Review defaulter list.

Attendance validation pseudocode:
IF user is not authenticated OR role is not Student: reject.
IF current time is outside configured attendance window: reject.
IF a record already exists for date + subject + roll: reject duplicate.
ELSE save attendance with verified = false and markedTime = current time.
CR verification pseudocode:

IF current user is not authorised CR: reject.
IF pending record exists: set verified = true, verifiedBy = CR roll, verifiedAt = current time.
Store all corrections with an audit trail.

## 13. SECURITY AND ANTI-PROXY MEASURES

Time Window: Attendance submission is enabled only during the configured class window, for example 9:00 AM to 4:15 PM. A grace period can be configured for individual classes. The server should determine the authoritative time.

One Student–Subject–Date Record: A deterministic record key and transaction-safe write prevent duplicate entries for the same student, subject and date.

One Device / Session (Optional): A session can limit repeated account switching in the same browser, but browser session controls are not strong identity proof and can be cleared or bypassed.

CR Physical Verification: The CR checks the class and confirms attendance. This is the main human verification safeguard in this no-biometric design.

Email OTP (Optional): A random six-digit code may be sent to the registered college email and expire after a short period. OTPs must be hashed or securely stored, rate-limited and never logged in plain text. Email OTP confirms access to an email account, not physical presence.

Location Check (Optional): Browser location may be used as a weak additional signal with explicit user permission. It can be spoofed, denied or inaccurate, so it must not be the sole basis for attendance.

Access Control and Privacy: Use role-based access, HTTPS, Firestore Security Rules, secure secrets, minimal personal data, audit logs and regular backups. Do not expose student data to unauthorised users.

## 14. USER INTERFACE DESIGN

The Student Home page contains a welcome message, current date, subject dropdown, Present/Absent selection and Mark Attendance button. It also displays submission status and previous attendance where permitted.

The CR Verification page contains filters and a pending table with Verify and Mark Absent actions. Every action records the CR identity and timestamp.

The Teacher Dashboard contains date, subject and semester filters; summary metrics; attendance percentage tables; a daily trend chart; a subject-wise bar chart; and report-generation controls.

The interface should use clear labels, readable colours, confirmation messages and responsive layouts. Do not rely on colour alone to communicate attendance status.

## 15. TESTING AND TEST CASES

Testing should be performed with test accounts and sample data before real student records are entered. Suggested test cases:

TC01 – Valid registration: enter unique roll number and valid details; expected result: account and profile created.
TC02 – Duplicate roll number: register an existing roll; expected result: registration rejected.
TC03 – Valid login: use correct credentials; expected result: correct dashboard opens.
TC04 – Invalid login: use incorrect credentials; expected result: access denied with safe message.
TC05 – In-window attendance: submit during permitted time; expected result: pending record saved.
TC06 – Late attendance: submit outside time window; expected result: submission rejected.
TC07 – Duplicate attendance: submit again for same date and subject; expected result: duplicate rejected.
TC08 – CR verification: authorised CR verifies pending entry; expected result: verified flag and verifier timestamp saved.

TC09 – Unauthorised verification: student attempts CR action; expected result: access denied.
TC10 – Report generation: apply filters and export; expected result: report matches stored records.
TC11 – Attendance threshold: student below configured percentage; expected result: appears in defaulter list.
TC12 – Database unavailable: attempt operation during outage; expected result: graceful error and no false success message.

Record actual test date, tester, input, expected result, actual result and Pass/Fail in the final submission after testing the working application.

## 16. REPORTS AND AI INSIGHTS

Attendance Percentage = (Number of Present classes ÷ Total conducted classes) × 100. The denominator should count classes actually conducted for the selected subject and date range; cancelled classes should not be counted.

A defaulter can be identified when overall attendance is below the institutional threshold or when a subject-wise percentage is below its threshold. The project brief uses 75% as an example. The final threshold must follow the college's current attendance policy.

Suggested charts include daily attendance trend (line chart), subject-wise attendance percentage (bar chart) and present/absent distribution. AI prompts should use aggregated or pseudonymised data where possible and ask for factual patterns, not unsupported predictions. A teacher must review all AI-generated observations.

## 17. DEPLOYMENT PLAN

1. Create a Firebase project and enable Firebase Authentication.
2. Create a Cloud Firestore database and configure collections and Security Rules.
3. Develop and test the Streamlit application locally.
4. Keep Firebase credentials and optional API keys in Streamlit Secrets or environment variables.
5. Add app.py and requirements.txt to a GitHub repository. Do not upload private keys.
6. Connect the repository to Streamlit Community Cloud and deploy the application.
7. Test registration, student marking, CR verification, teacher dashboard and exports with dummy users.
8. Share the live URL only after confirming access controls and sample workflows.

Suggested repository structure:
attendance-system/
app.py
firebase_config.py
requirements.txt
README.md
.gitignore
pages/
student.py
cr_verification.py
teacher_dashboard.py
utils/
attendance.py
reports.py
assets/
.streamlit/
secrets.toml (local only; never commit)

## 18. EXPECTED RESULTS

When implemented and tested, the system is expected to provide a centralised attendance record, role-based student/CR/teacher views, duplicate prevention, pending verification, attendance percentage calculations, defaulter lists and downloadable reports. It should reduce repetitive manual calculations and make attendance information easier to review.

Actual screenshots, deployment URL, measured test results and demonstration video should be added after the application has been built. This report describes the design and intended behaviour; it does not claim that a live application has already been deployed.

## 19. FUTURE ENHANCEMENTS

Possible future improvements include QR-code based classroom check-in, integration with the college ERP, timetable-based automatic subject selection, mobile-friendly notifications, stronger administrator controls, scheduled backups, multi-class support and improved audit logs. Face recognition or GPS-based features may be considered only if the institution approves them and privacy, consent, accuracy and legal requirements are addressed.

## 20. CONCLUSION

The Student Attendance Management System provides a practical approach to digitising classroom attendance without face recognition or biometric hardware. Student self-submission, CR verification and teacher dashboards form the core workflow. Time restrictions and duplicate checks discourage common misuse, while reports and charts make attendance trends easier to understand.

The system's effectiveness depends on correct configuration, secure role permissions, reliable data and responsible human verification. With these safeguards and proper testing, the project can serve as a useful BCA-level demonstration of web development, database management and software engineering concepts.

## 21. REFERENCES

1. Python Documentation – https://docs.python.org/3/
2. Streamlit Documentation – https://docs.streamlit.io/
3. Firebase Documentation – https://firebase.google.com/docs
4. Cloud Firestore Documentation – https://firebase.google.com/docs/firestore
5. Pandas Documentation – https://pandas.pydata.org/docs/
6. Plotly Python Documentation – https://plotly.com/python/
7. ReportLab Documentation – https://docs.reportlab.com/
8. Streamlit Community Cloud – https://docs.streamlit.io/deploy/streamlit-community-cloud

Accessed for project planning and technology reference. Verify current service limits, security guidance and pricing before deployment.

## 22. APPENDIX A – SAMPLE FIRESTORE STRUCTURE

students/{roll}
name: "Student Name"
course: "BCA"
semester: 3
email: "student@example.edu"
authUid: "Firebase Authentication UID"
role: "student"

subjects/{subjectId}
subjectName: "Database Management Systems"
teacherName: "Faculty Name"
semester: 3
active: true

attendance/{YYYY-MM-DD_subjectId_roll}
date: "2026-11-09"
subjectId: "SUB001"
roll: "BCA001"
status: "Present"
markedTime: "2026-11-09T09:10:00"
verified: false
verifiedBy: null
verifiedAt: null

These are illustrative field examples. Adapt field names and document IDs to the final application and enforce validation in the application and database rules.

## 23. APPENDIX B – SAMPLE PYTHON LOGIC

The following simplified example demonstrates the validation sequence. It is illustrative pseudocode and must be connected to authenticated user data, server-side time and a transaction-safe Firestore write before production use.

```
def can_mark_attendance(user, subject_id, today, now, existing_record):
if not user or user.get("role") != "student":
return False, "Student login required"

if not (CLASS_START <= now <= CLASS_END):
return False, "Attendance window is closed"

if existing_record:
return False, "Attendance already submitted"

return True, "Allowed"

# After validation, create a pending record:
# {
# "date": today,
# "subjectId": subject_id,
# "roll": authenticated_student_roll,
# "status": selected_status,
# "markedTime": server_timestamp,
# "verified": False
# }
```

## 24. APPENDIX C – VIVA QUESTIONS AND ANSWERS

Q1. What is the main purpose of this project?
A. To digitise attendance entry, verification, monitoring and report generation.

Q2. Why is face recognition not used?
A. The project is designed to be simple and low-cost without biometric hardware or face data.

Q3. What is Firebase Authentication?
A. A managed service used to create accounts and authenticate users.

Q4. What is Cloud Firestore?
A. A cloud-hosted NoSQL database that stores application records in collections and documents.

Q5. Why is CR verification required?
A. It adds a human check that the student-submitted attendance corresponds to the class.

Q6. How are duplicate entries prevented?
A. By checking for an existing date-subject-student record and using a deterministic document ID with a transaction-safe write.

Q7. How is attendance percentage calculated?
A. Present classes divided by total conducted classes, multiplied by 100.

Q8. Can the system completely prevent proxy attendance?
A. No. Without biometrics it cannot guarantee physical identity. Time controls, authentication and CR verification reduce risk but do not eliminate it.

Q9. What is the role of Streamlit?
A. It provides the web interface for forms, tables, dashboards and charts.

Q10. What are the future enhancements?
A. QR check-in, ERP integration, notifications, stronger administration and optional approved identity/location features.

## PROJECT SUBMISSION CHECKLIST

- Replace all blank student/college/guide details.
- Add screenshots of the completed application.
- Add the actual GitHub repository and live deployment URLs.
- Run the test cases and fill in actual Pass/Fail results.
- Attach the 2-minute demonstration video link if required.
- Confirm attendance threshold and college policy with the department.